

PKCERT AI & Software Development Internship

Task 03 — Python Fundamentals & Introduction to AI/ML

Abdullah Amir

AI and Software Development
06 July 2026

Contents

Part A — Python Fundamentals	1
Part B — Python for Data Handling	3
Part C — Development Environment	4
Part D — Introduction to AI & Machine Learning	5
Part E — AI Libraries Research	6

Part A — Python Fundamentals

1. Python Data Types and Their Uses

One thing I like about Python is that you don't declare a type up front. You just assign a value and Python figures out the rest at runtime. Still, it helps to know the built-in types and when each one makes sense:

- **int** — plain whole numbers like `age = 22`. Good for counting and indexing.
- **float** — numbers with a decimal point, such as `price = 99.5`. I reach for these whenever there's a measurement or a calculation that won't stay whole.
- **str** — text. Strings can't be changed in place once created.
- **bool** — just `True` or `False`, which is what drives most `if` statements.
- **list** — an ordered collection you can edit, like `[1, 2, 3]`.
- **tuple** — also ordered, but fixed once made. Handy for things that shouldn't change, like a coordinate pair.
- **dict** — key-value pairs. This is my go-to when I need to look things up quickly by name.
- **set** — keeps only unique items, so it's an easy way to drop duplicates.

```
1 age = 22                # int
2 price = 99.5            # float
3 name = "Abdullah"       # str
4 is_active = True        # bool
5 skills = ["Python", "AI"] # list
6 point = (10.0, 20.0)     # tuple (can't be changed)
7 student = {"id": 1, "name": name} # dict
8 unique_ids = {1, 2, 2, 3} # set -> {1, 2, 3}
```

```

9
10 for value in (age, price, name, is_active, skills, point, student,
    unique_ids):
11     print(type(value).__name__, "->", value)

```

Listing 1: Trying out each of the core data types.

2. User-Defined Functions with Parameters and Return Values

Functions are how I avoid writing the same thing twice. You pass values in through parameters and get something back with `return`. Default arguments are also useful when a parameter usually has one obvious value.

```

1 def calculate_area(length, width=1):
2     """Area of a rectangle. If no width is given, assume 1."""
3     return length * width
4
5 def grade(marks):
6     """Turn a numeric mark into a letter grade."""
7     if marks >= 80:
8         return "A"
9     elif marks >= 60:
10        return "B"
11    return "C"
12
13 print(calculate_area(5, 4))    # 20
14 print(calculate_area(7))      # 7 (width defaults to 1)
15 print(grade(85))              # A

```

Listing 2: Functions with parameters, a default argument, and return values.

3. List and Dictionary Comprehensions

Comprehensions took me a moment to get used to, but now I prefer them. They let you build a list or dictionary in a single line instead of setting up an empty container and appending inside a loop.

```

1 # squares of the even numbers from 0 to 9
2 squares = [n * n for n in range(10) if n % 2 == 0]
3 # -> [0, 4, 16, 36, 64]
4
5 # map each number to its cube
6 cubes = {n: n ** 3 for n in range(1, 6)}
7 # -> {1: 1, 2: 8, 3: 27, 4: 64, 5: 125}
8
9 print(squares)
10 print(cubes)

```

Listing 3: A list comprehension and a dictionary comprehension.

The benefit is mostly readability: the whole intent sits on one line, so anyone reading it can see straight away what the collection contains. They also tend to run a little faster than the loop-and-append version because Python handles the iteration internally.

4. Object-Oriented Programming: Classes, Objects, Constructors, and Inheritance

Object-oriented programming is basically about modelling things as objects. You write a class as a blueprint, the `__init__` method sets up each object when it's created, and inheritance lets

one class build on top of another instead of repeating code.

```
1 class Person:
2     def __init__(self, name, age):      # constructor
3         self.name = name
4         self.age = age
5
6     def introduce(self):
7         return f"Hi, I am {self.name}, aged {self.age}."
8
9 class Student(Person):                 # Student builds on Person
10    def __init__(self, name, age, roll_no):
11        super().__init__(name, age)    # reuse the parent's setup
12        self.roll_no = roll_no
13
14    def introduce(self):                # override with extra detail
15        base = super().introduce()
16        return f"{base} My roll number is {self.roll_no}."
17
18 p = Person("Ali", 30)
19 s = Student("Abdullah", 22, "F21-001")
20 print(p.introduce())
21 print(s.introduce())
```

Listing 4: A base class and a subclass that inherits from it.

This small example ends up covering everything the task asked for. `Person` and `Student` are the classes, `p` and `s` are objects made from them, `__init__` is the constructor, and `Student` inherits from `Person` while overriding `introduce()` to add its own line.

Part B — Python for Data Handling

1. Reading, Writing, and Appending Text Files

For files I stick to `open()` inside a `with` block, since that closes the file for me even if something goes wrong. The mode is what decides the behaviour: `"w"` writes from scratch (and wipes whatever was there), `"a"` adds to the end, and `"r"` just reads.

```
1 # write - creates the file or overwrites it
2 with open("notes.txt", "w") as f:
3     f.write("Line 1: Python fundamentals\n")
4
5 # append - keeps the old content and adds to the end
6 with open("notes.txt", "a") as f:
7     f.write("Line 2: File handling\n")
8
9 # read it all back
10 with open("notes.txt", "r") as f:
11     print(f.read())
```

Listing 5: Writing, then appending, then reading a text file.

2. Exception Handling

Instead of letting the program crash on a bad input, I wrap the risky part in a `try` block and deal with the specific errors in `except`. The `finally` block runs no matter what, which is handy for cleanup or a closing message.

```

1 def safe_divide(a, b):
2     try:
3         result = a / b
4     except ZeroDivisionError:
5         print("Error: cannot divide by zero.")
6         result = None
7     except TypeError:
8         print("Error: inputs must be numbers.")
9         result = None
10    finally:
11        print("Division attempt finished.")
12    return result
13
14 print(safe_divide(10, 2))    # 5.0
15 print(safe_divide(10, 0))    # handled -> None

```

Listing 6: Catching errors with try / except / finally.

3. Reading and Displaying CSV and JSON Files

Python already ships with modules for both of these, so I didn't need to install anything. CSV is for row-and-column data, while JSON is the nested format you usually get back from web APIs.

```

1 import csv
2 import json
3
4 # --- CSV ---
5 with open("students.csv", newline="") as f:
6     reader = csv.DictReader(f)          # uses the first row as column
7     names                                     names
8     for row in reader:
9         print(row["name"], "-", row["marks"])
10
11 # --- JSON ---
12 with open("config.json") as f:
13     data = json.load(f)                  # comes back as a normal dict
14     print(data["project"], "v", data["version"])

```

Listing 7: Reading a CSV file and a JSON file.

Part C — Development Environment

Virtual Environments, Conda Environments, and Dependency Isolation

The problem this part is really about shows up once you have more than one project on the same laptop. Two projects might need different versions of the same package, and if you install everything globally they start stepping on each other. The fix is to give every project its own isolated set of packages.

Virtual environments (venv). Python already includes the `venv` module for this. It creates a small folder with its own copy of the interpreter and its own **site-packages**, so anything you `pip install` while it's active only affects that one project.

```

1 python -m venv venv          # create it
2 venv\Scripts\activate        # activate (Windows)

```

```

3 pip install numpy pandas          # only goes into this env
4 pip freeze > requirements.txt
5 deactivate                        # step back out

```

Listing 8: Setting up and using a virtual environment.

Conda environments. Conda comes with Anaconda or Miniconda and does a bit more than `venv`. It isn't limited to Python packages, so it can also pull in system-level things like CUDA, and each environment can even run a different Python version altogether.

```

1 conda create -n ml_env python=3.11 # pick the Python version here
2 conda activate ml_env
3 conda install numpy scikit-learn
4 conda env export > environment.yml
5 conda deactivate

```

Listing 9: Setting up and using a Conda environment.

So why bother with isolation at all? Mostly it keeps your global Python clean and stops the classic “it works on my machine” problem. Because you can export the exact package list to a `requirements.txt` or `environment.yml`, a teammate can recreate the same setup and get the same results. The way I think about it: `venv` is the lightweight, Python-only option, while Conda is heavier but handles system libraries and multiple Python versions, which is why it's so common in data-science and AI work.

Part D — Introduction to AI & Machine Learning

1. What is Artificial Intelligence?

Artificial Intelligence is the part of computer science that tries to get machines to do things we'd normally call intelligent — reasoning, learning from experience, understanding language, recognising images, and making decisions. Put simply, the goal is to make a computer act in a way that looks smart to us.

2. AI vs. Machine Learning vs. Deep Learning

People often use these three words interchangeably, but they actually sit inside one another. Table 1 lays out how they differ.

Field	What it means
Artificial Intelligence (AI)	The widest term. Any method that lets a machine imitate human intelligence, including old-school rule-based systems.
Machine Learning (ML)	A part of AI where the system learns patterns from data on its own, instead of being told every rule by hand.
Deep Learning (DL)	A part of ML that stacks many layers of artificial neural networks to learn from huge datasets like images or audio.

Table 1: How AI, ML, and DL nest inside each other ($\text{AI} \supseteq \text{ML} \supseteq \text{DL}$).

3. Types of Machine Learning

- **Supervised learning** works with labelled data, meaning every input already has a known answer attached. Spam filters, house-price prediction, and image classification all fall here.

- **Unsupervised learning** is given data with no labels and has to find structure by itself. Grouping customers into segments or reducing the number of features are typical uses.
- **Reinforcement learning** is different again: an agent tries actions in some environment and learns from the rewards or penalties it gets. Game-playing bots like AlphaGo, robotics, and self-driving control are the usual examples.

4. The Machine Learning Pipeline

Most ML projects follow roughly the same set of steps, even if the details change:

1. **Collect the data** from files, databases, sensors, or APIs.
2. **Preprocess it** — clean it up, fill in or drop missing values, encode categories, and scale the numbers.
3. **Work on the features**, creating and picking the ones that actually carry useful signal.
4. **Choose a model** that suits the problem, whether that's regression, classification, or clustering.
5. **Train it** on the training data so it learns the patterns.
6. **Evaluate it** on data it hasn't seen, using metrics like accuracy or RMSE.
7. **Tune the hyper-parameters** to squeeze out better performance.
8. **Deploy and monitor** the model in production, retraining it when new data shows up.

Part E — AI Libraries Research

These five libraries come up constantly in Python AI work, and they each fill a different slot. Table 2 sums up what they're for.

Library	Purpose	Where it's used
NumPy	Fast numerical computing built around N-dimensional arrays and vectorised math.	Linear algebra and array work; it sits underneath most of the other libraries.
Pandas	Data analysis using <code>DataFrame</code> and <code>Series</code> objects.	Loading, cleaning, filtering, and summarising tabular data from CSV, Excel, or SQL.
Scikit-learn	A classic machine-learning toolkit with a clean, consistent API.	Regression, classification, clustering, preprocessing, and model evaluation.
TensorFlow	Google's deep-learning framework, with strong support for deployment.	Neural networks, computer vision, NLP, and large-scale training and serving.
PyTorch	Meta's deep-learning framework, popular in research for its dynamic graph.	Deep-learning research, computer vision, NLP, and quick prototyping.

Table 2: The main Python AI/ML libraries at a glance.